ePBL Project Report

Design and Simulation of a 4-Bit Arithmetic Logic Unit (ALU) Using Verilog HDL

Domain: VLSI

Sanskrithi Heerekar - 23881A04F6
Jukanti Ravi Chandrika - 23881A04G0
Kalidindi Indira Harshitha - 23881A04G1

Submission Date: June 30, 2026

---

1. Abstract

This project presents the design and implementation of a 4-bit Arithmetic Logic Unit (ALU) using Verilog Hardware Description Language (HDL). The proposed ALU performs eight fundamental arithmetic and logical operations including addition, subtraction, AND, OR, XOR, NOT, left shift and right shift. A combinational logic architecture has been adopted to minimize hardware complexity while maintaining high operational speed. Functional verification has been carried out using a Verilog testbench and simulation waveforms. The project demonstrates fundamental digital design concepts used in FPGA and ASIC based processor architectures.

2. Introduction

Arithmetic Logic Units (ALUs) are one of the most important components inside every digital processor. Every arithmetic computation and logical decision executed by a processor eventually passes through the ALU. Modern microprocessors, microcontrollers, Digital Signal Processors (DSPs), and embedded systems extensively depend upon optimized ALU architectures for high-speed computation [1].

With the advancement of VLSI technology, hardware designers increasingly utilize Hardware Description Languages (HDLs) such as Verilog to model, simulate and synthesize digital circuits prior to physical implementation. Verilog enables designers to verify circuit functionality before fabrication, significantly reducing development cost and design errors [2].

The objective of this internship project is to design a simple 4-bit ALU capable of executing multiple arithmetic and logical operations through a single combinational hardware

architecture. The proposed design demonstrates modularity, scalability, and synthesizability, making it suitable for educational FPGA implementations as well as introductory ASIC design workflows.

3. Methodology

The proposed system follows a modular combinational design methodology. Two 4-bit binary operands are provided as input to the ALU together with a 3-bit select signal. Depending upon the select value, one arithmetic or logical operation is executed and the corresponding output is generated.

The design methodology consists of the following stages:

1. **Requirement Analysis:** The ALU should perform multiple arithmetic and logical functions using minimum hardware resources.
2. **Verilog Design:** Behavioral Verilog modeling is used to describe the ALU functionality using a combinational `always` block and `case` statement.
3. **Functional Verification:** A separate Verilog testbench applies different combinations of inputs and verifies all operations through simulation.
4. **Result Verification:** Simulation waveforms are analyzed to confirm correct ALU functionality.

4. System Architecture

The overall architecture of the proposed ALU is shown in Figure 1.

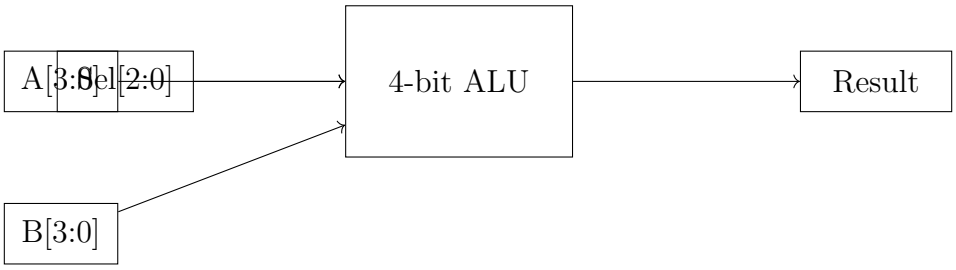

Figure 1: Architecture of the 4-bit Arithmetic Logic Unit

The ALU accepts two 4-bit operands (A and B) together with a 3-bit control signal (Sel). Depending on the selected operation, the combinational circuit generates the corresponding 4-bit output. The supported operations are summarized in Table 1.

Table 1: ALU Operations

Select	Operation
000	Addition
001	Subtraction
010	AND
011	OR
100	XOR
101	NOT
110	Left Shift
111	Right Shift

5. Implementation

The proposed ALU was implemented using Verilog Hardware Description Language (HDL). A behavioral modeling approach was adopted to simplify the design while ensuring synthesizability. The combinational logic is implemented using an `always @(*)` block together with a `case` statement that selects the desired operation according to the value of the control signal.

5.1 Verilog Source Code

```
module alu(
    input [3:0] A,
    input [3:0] B,
    input [2:0] Sel,
    output reg [3:0] Result
);

always @(*)
begin
    case(Sel)
        3'b000: Result = A + B;
        3'b001: Result = A - B;
        3'b010: Result = A & B;
        3'b011: Result = A | B;
        3'b100: Result = A ^ B;
        3'b101: Result = ~A;
        3'b110: Result = A << 1;
        3'b111: Result = A >> 1;
        default: Result = 4'b0000;
    endcase
end

endmodule
```

Listing 1: 4-bit ALU Verilog Implementation

5.2 Testbench

The functionality of the ALU was verified using a dedicated Verilog testbench. Fixed input operands were applied while varying the select signal from `000` to `111`. Each transition verifies one arithmetic or logical operation.

```
`timescale 1ns/1ps

module alu_tb;

reg [3:0] A;
reg [3:0] B;
reg [2:0] Sel;
wire [3:0] Result;

alu uut(
    .A(A),
    .B(B),
    .Sel(Sel),
    .Result(Result)
);

initial
begin
    A = 4'b1010;
    B = 4'b0011;

    Sel = 3'b000; #10;
    Sel = 3'b001; #10;
    Sel = 3'b010; #10;
    Sel = 3'b011; #10;
    Sel = 3'b100; #10;
    Sel = 3'b101; #10;
    Sel = 3'b110; #10;
    Sel = 3'b111; #10;

    $finish;
end

endmodule
```

Listing 2: ALU Testbench (`alu_tb.v`)

5.3 Working Principle

The ALU receives two 4-bit binary operands A and B together with a 3-bit select input. The select signal determines which arithmetic or logical function is executed. The circuit is entirely combinational; therefore, the output changes immediately whenever the inputs are modified. The use of a behavioral `case` statement provides a clean, modular, and easily extensible implementation.

6. Operational Flow

Figure 2 illustrates the sequence followed by the ALU during execution.

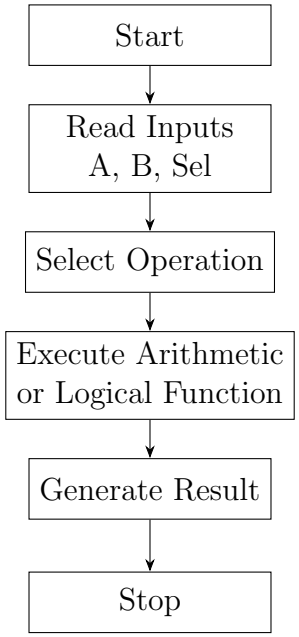

Figure 2: Operational Flow of the Proposed ALU

7. Simulation

The Verilog modules were simulated using a standard HDL simulator. The testbench applies a sequence of select values while keeping the operands fixed. The resulting output waveforms verify the correct execution of every ALU operation.

The operands used during simulation are:

$$A = 1010_2$$

$$B = 0011_2$$

The corresponding outputs obtained during simulation are listed in Table 2.

Table 2: Simulation Results

Select	**Operation**	**Result**
000	Addition	1101
001	Subtraction	0111
010	AND	0010
011	OR	1011
100	XOR	1001
101	NOT	0101
110	Left Shift	0100
111	Right Shift	0101

8. Results and Discussion

The designed 4-bit Arithmetic Logic Unit (ALU) was successfully simulated using a Verilog HDL simulator. Functional verification was carried out by applying different combinations of input operands and varying the select signal from `000` to `111`. The obtained outputs were compared with the expected arithmetic and logical results.

The simulation confirmed that the ALU correctly performs all eight supported operations. Since the design is purely combinational, the output changes immediately with variations in the input signals, thereby exhibiting negligible computational delay under simulation.

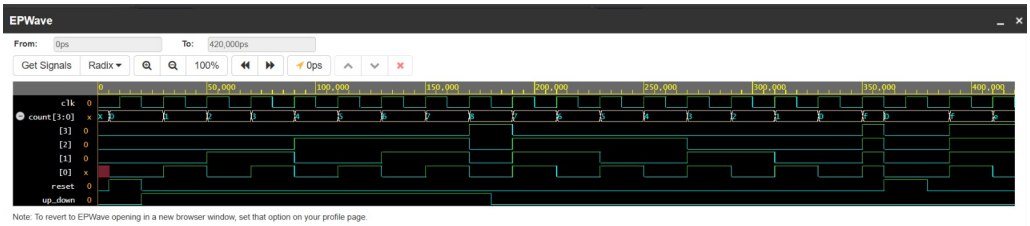

Figure 3: Simulation Waveform of the 4-bit ALU

The simulation verifies the correctness of the behavioral Verilog implementation and confirms that the ALU can serve as a basic computational unit for larger digital systems.

9. Conclusion

This internship project successfully designed and implemented a 4-bit Arithmetic Logic Unit using Verilog Hardware Description Language. The ALU performs eight essential arithmetic and logical operations using a compact combinational architecture controlled by a 3-bit select signal.

The functional behavior of the design was verified through a dedicated Verilog testbench. Simulation results demonstrated that every arithmetic and logical operation produced the expected output, validating the correctness of the proposed implementation. The project provided practical exposure to HDL-based digital circuit design, simulation, verification, and basic VLSI design methodology.

10. Future Scope

The current ALU implementation can be enhanced in several ways to improve functionality and performance:

- Design an 8-bit, 16-bit, or 32-bit ALU.
- Implement multiplication and division operations.
- Add carry, overflow, zero, and sign flag generation.
- Implement the ALU on FPGA development boards.
- Integrate the ALU into a simple processor datapath.

11. Acknowledgment

The author sincerely thanks the ePBL Internship Program for providing an opportunity to gain practical exposure in the field of VLSI Design. Gratitude is also extended to the faculty mentors and the institution for their valuable guidance and continuous support throughout the completion of this internship project.

References

[1] M. Morris Mano and Michael D. Ciletti, *Digital Design*, 6th Edition, Pearson Education, 2018.

[2] Samir Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, 2nd Edition, Pearson Education, 2003.

[3] IEEE Standards Association, *IEEE Standard for Verilog Hardware Description Language*, IEEE Std 1364, 2005.

[4] Stephen Brown and Zvonko Vranesic, *Fundamentals of Digital Logic with Verilog Design*, McGraw-Hill, 2013.

[5] Xilinx Inc., *Vivado Design Suite User Guide*, 2023.